

Vertex State Engine Specification

Version: 1.0

Status: Draft

1. Objetivo

O State Engine é o subsistema do Vertex responsável por gerenciar estados compartilhados da plataforma.

Seu objetivo é fornecer uma fonte única e consistente de informações que representam a situação atual do sistema, dos serviços e dos plugins.

Enquanto o Event System informa que algo aconteceu, o State Engine informa como algo está neste momento.

O State Engine é otimizado para consultas frequentes, observação de mudanças e sincronização eficiente.

2. Responsabilidades

O State Engine é responsável por:

- registrar estados;
- atualizar estados;
- disponibilizar estados para consulta;
- permitir observação de alterações;
- validar tipos;
- sincronizar mudanças;
- persistir estados quando necessário;
- controlar visibilidade;
- manter consistência.

O State Engine não executa lógica de negócio.

3. Arquitetura

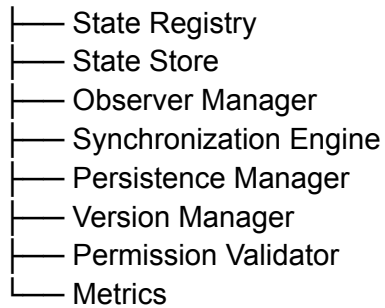
Plugin

↓

State API

↓

State Engine



Todo acesso aos estados ocorre através do State Engine.

4. Componentes

State Registry

Mantém o catálogo de todos os estados registrados.

Cada estado possui:

- ID;
- tipo;
- versão;
- proprietário;
- visibilidade;
- metadados.

State Store

Armazena os valores atuais dos estados.

É a fonte oficial de consulta.

Observer Manager

Gerencia observadores inscritos.

Sempre que um estado muda, os observadores são notificados.

Synchronization Engine

Coordena a propagação das alterações de estado.

Evita atualizações inconsistentes.

Persistence Manager

Responsável por persistir estados marcados como persistentes.

Estados temporários permanecem apenas em memória.

Version Manager

Controla compatibilidade entre diferentes versões de estados.

Permission Validator

Verifica permissões de leitura e escrita.

Metrics Collector

Coleta estatísticas de utilização.

5. Fluxos

Registro

Plugin

↓

registerState()

↓

Registry

Atualização

Plugin

↓

update()

↓

State Store

↓

Synchronization

↓

Observers

Consulta

Plugin

↓

get()

↓

State Store

↓

Resultado

Observação

Plugin

↓

observe()

↓

Observer Manager

↓

Atualizações

6. APIs

Registro

- registerState()
- unregisterState()

Consulta

- get()
- exists()
- metadata()

Atualização

- update()
- compareAndSet()

Observação

- observe()
- removeObserver()

Persistência

- save()
- restore()

7. Estados

Estado do registro

Registered

↓

Available

↓

Updating

↓

Synchronized

↓

Persisted

Estados adicionais:

- Removed;
- Invalid.

8. Políticas

Fonte Única da Verdade

Cada estado possui apenas um valor oficial.

Tipagem

Todos os estados devem possuir tipos definidos.

Atualização Controlada

Estados somente podem ser modificados pelo proprietário autorizado ou por serviços autorizados.

Persistência Opcional

Cada estado define se será:

- temporário;
- persistente.

Observação

Observadores recebem apenas mudanças efetivas de valor, evitando notificações desnecessárias.

Consistência

Atualizações concorrentes devem ser tratadas de forma determinística.

Segurança

Leitura e escrita respeitam as permissões definidas pelo Security Framework.

9. Integrações

O State Engine integra-se com:

- Plugin Manager;
- Plugin API;
- Plugin Context;
- Event System;
- Capability System;
- Scheduler;
- Resource Manager;
- Storage Engine;
- Security Framework;

- Policy Engine;
- Configuration System;
- Logging Framework;
- Memory Management;
- Plugin Inspector.

O State Engine é complementar ao Event System.

10. Métricas

O State Engine registra:

- estados registrados;
- estados persistentes;
- atualizações realizadas;
- consultas executadas;
- observadores ativos;
- tempo médio de atualização;
- tempo médio de leitura;
- conflitos de atualização;
- falhas de sincronização;
- consumo de memória pelos estados.

Essas métricas auxiliam otimizações e diagnósticos.

11. Exemplos

Estado da bateria

battery.level

Consumido por:

- Watchfaces;
- Power Management;
- Widgets.

Estado da reprodução

media.playback

Consumido por:

- Controles rápidos;
- Notificações;
- Watchface.

Estado do tema

system.theme

Consumido por todos os componentes de interface.

Estado da conexão

phone.connection

Consumido por:

- Plugin de conexão;
- Interface;
- Sincronização.

12. Regras Arquiteturais

- Estados representam o presente.
- Eventos representam acontecimentos passados.
- Capabilities representam funcionalidades.
- Apenas o State Engine pode armazenar estados compartilhados.
- Plugins nunca compartilham estados diretamente.
- Observadores devem ser leves e não bloquear atualizações.
- O acesso aos estados deve ocorrer exclusivamente pela State API.

13. Limitações

O State Engine não é responsável por:

- executar funcionalidades;
- distribuir eventos;
- armazenar arquivos;
- controlar ciclo de vida;
- gerenciar permissões de plugins.

Essas funções pertencem a outros subsistemas.

14. Futuras Extensões

O State Engine poderá evoluir para suportar:

- estados derivados (computed states);
- sincronização entre múltiplos dispositivos;
- histórico opcional de estados;
- atualização em lote (batch updates);
- transações atômicas entre múltiplos estados;
- cache inteligente de leitura.

Essas extensões devem preservar o princípio da fonte única da verdade e manter baixo consumo de recursos.

15. Resumo

O State Engine é o repositório central de estados compartilhados do Vertex. Ele fornece uma visão consistente do estado atual da plataforma, permitindo consultas rápidas, observação eficiente de mudanças e persistência quando necessário. Em conjunto com o Event System e o Capability System, forma um modelo de comunicação claro: Capabilities executam ações, Events notificam acontecimentos e States representam a condição atual do sistema.